

# CANLIFAL FLUTTER — CANLI YAYIN SİSTEMİ TAM ENTEGRASYON PROMPT'U

**Base URL:** `https://canlifal.com`

**Auth Header:** `Authorization: Bearer {JWT}` (tüm isteklerde)

**Agora App ID:** `f1cf983a38114b04a4e9102c303ba63e`

## 1. CANLI YAYIN AÇMA (Broadcaster Ekranı)

### 1.1 Yayın Oluşturma

```
POST /api/video-streams
Headers: Authorization: Bearer {JWT}
Body: {
  "title": "Canlı Fal",
  "description": "Kahve falı bakıyorum",
  "category": "fortune", // fortune | chat | general
  "tags": "fal,kahve",
  "thumbnailUrl": "https://...", // opsiyonel
  "coverUrl": "https://..." // opsiyonel
}
Response: {
  "success": true,
  "data": {
    "id": "clxyz...",
    "streamId": "clxyz...",
    "roomId": "auto-generated-unique-id",
    "title": "Canlı Fal",
    "status": "live",
    "isLive": true,
    "broadcasterId": "user-id",
    "hostUserId": "user-id",
    "streamerName": "Falcı Ayşe",
    "viewerCount": 0
  }
}
```

## 1.2 Agora Token Alma (Yayını Başlatmadan Önce)

```
POST /api/agora/token
Headers: Authorization: Bearer {JWT}
Body: {
  "channelName": "{streamId}", // veya roomId kullanılabilir
  "role":
  "host", // "host" = yayıncı (PUBLISHER), "audience" = izleyici (SUB-SCRIBER)
  "uid": 0 // 0 = auto-assign, veya sabit numeric UID
}
Response: {
  "token": "006f1cf983...",
  "uid": 0,
  "channelName": "clxyz...",
  "appId": "f1cf983a38114b04a4e9102c303ba63e"
}
```

## 1.3 Yayın Canlıya Geçti Bildirimi (Agora bağlandıktan SONRA çağır)

```
POST /api/video-streams/{streamId}/live-started
Headers: Authorization: Bearer {JWT}
Body: {}
Response: { "success": true, "notifiedCount": 42 }
```

Takipçilere OneSignal push notification + in-app bildirim gönderir (max 500 takipçi).

## 1.4 Yayın Güncelleme

```
PATCH /api/video-streams/{streamId}
Headers: Authorization: Bearer {JWT}
Body: {
  "title": "Yeni Başlık",
  "description": "...",
  "broadcastImage": "https://upload.wikimedia.org/wikipedia/commons/thumb/8/88/Nintendo-Switch-wJoyCons-BLRd-Standing-FL.jpg/500px-Nintendo-Switch-wJoyCons-BLRd-Standing-FL.jpg", // resimli yayın modu
  "isImageMode": true, // true = video yerine resim göster
  "backgroundUrl": "https://www.axeltechnology.com/wp-content/uploads/2023/04/Axel-Tech-XTV-XPlayout-Main-Screen-Graphic-uai-1777x1000.jpg" // arka plan resmi
}
```

## 1.5 Yayın Sonlandırma

```
POST /api/video-streams/{streamId}/end
Headers: Authorization: Bearer {JWT}
Body: {}
Response: { "success": true }
```

Tüm izleyicilere SSE üzerinden `streamEnded` event'i gönderilir. Tüm viewer'lar otomatik `leftAt` ile işaretlenir.

## 2. CANLI YAYIN İZLEME (Viewer Ekranı)

### 2.1 Yayın Listesi

```
GET /api/video-streams?page=1&limit=30
Headers: Authorization: Bearer {JWT} // opsiyonel
Response: {
  "streams": [
    {
      "id": "...",
      "streamId": "...",
      "title": "Canlı Fal",
      "status": "live",
      "isLive": true,
      "viewers": 15,
      "watching": 15,
      "broadcasterId": "user-id",
      "hostUserId": "user-id",
      "streamerName": "Falçı Ayşe",
      "thumbnailUrl": "...",
      "coverUrl": "...",
      "isImageMode": false,
      "broadcastImage": null,
      "backgroundUrl": null,
      "category": "fortune",
      "user": { "id": "...", "name": "...", "image": "..." }
    }
  ],
  "items": [...], // streams ile aynı (Flutter uyumluluğu)
  "pagination": { "page": 1, "limit": 30, "total": 5 }
}
```

NOT: Response bir object döner, array değil. `data.streams` veya `data.items` kullanın.

### 2.2 Tek Yayın Detay

```
GET /api/video-streams/{streamId}
Headers: Authorization: Bearer {JWT} // opsiyonel
Response: {
  "id": "...",
  "streamId": "...",
  "title": "...",
  "status": "live",
  "isLive": true,
  "viewers": 15,
  "watching": 15,
  "broadcasterId": "user-id",
  "hostUserId": "user-id",
  "streamerName": "...",
  "thumbnailUrl": "...",
  "coverUrl": "...",
  "isImageMode": false,
  "broadcastImage": null,
  "backgroundUrl": null,
  "user": { "id": "...", "name": "...", "image": "..." }
}
```

## 2.3 Yayına Katıl (Viewer)

```
POST /api/video-streams/{streamId}/join
Headers: Authorization: Bearer {JWT} // opsiyonel, yoksa guest_xxx ID atanır
Body: {}
Response: { "viewerId": "...", "joined": true, "viewerCount": 16 }
```

## 2.4 Yayından Ayrıl

```
POST /api/video-streams/{streamId}/leave
Headers: Authorization: Bearer {JWT}
Body: { "viewerId": "user-id" } // opsiyonel, auth varsa otomatik alınır
Response: { "left": true, "viewerCount": 15 }
```

Alternatif: `DELETE /api/video-streams/{streamId}/join?viewerId=xxx`

## 2.5 İzleyici Listesi

```
GET /api/video-streams/{streamId}/viewers
Headers: Authorization: Bearer {JWT} // opsiyonel
Response: [
  {
    "id": "user-id",
    "odUserId": "user-id",
    "name": "Ali",
    "image": "https://pbs.twimg.com/media/G46rKaSWUAAu2Rq.jpg",
    "hasGifted": true,
    "totalGiftAmount": 5000
  },
  {
    "id": "user-id-2",
    "odUserId": "user-id-2",
    "name": "Veli",
    "image": null,
    "hasGifted": false,
    "totalGiftAmount": 0
  }
]
```

Hediye gönderenler listede önce gelir, toplam hediye miktarına göre sıralı.

## 2.6 SSE Real-Time Events (Canlı Akış)

```
GET /api/video-streams/{streamId}/stream
Headers: Authorization: Bearer {JWT} // opsiyonel
Content-Type: text/event-stream (response)
```

**Bağlantı kurulunca gelen ilk event:**

```
{"type": "connected", "streamId": "..."}

```

**Ardından gelen event tipleri:**

```
// İzleyici sayısı değişimi
{"type": "viewerCount", "streamId": "...", "viewerCount": 15, "viewers": 15}

// Yeni chat mesajı
{"type": "streamMessage", "streamId": "...", "message": {
  "id": "...",
  "streamId": "...",
  "content": "merhaba",
  "createdAt": "2025-01-01T00:00:00.000Z",
  "user": {
    "id": "...",
    "name": "Ali",
    "nickname": null,
    "image": "https://i.pinimg.com/736x/18/db/33/18db3321356b96f8d72fac1dcf47bec0.jpg",
    "role": "free",
    "membership": null
  }
}}

// Hediye gönderildi
{"type": "gift", "streamId": "...", "gift": {
  "id": "...",
  "senderName": "Ali",
  "giftName": "Gül",
  "giftIcon": "🌹",
  "quantity": 1,
  "totalPrice": 100
}}

// Yayın sonlandı
{"type": "streamEnded", "event": "STREAM_ENDED", "streamId": "..."}

```

**Heartbeat:** Her 15 saniyede `: heartbeat\n\n` gönderilir (keep-alive).

**Poll interval:** Server 1 saniyede bir yeni event kontrol eder.

## 2.7 Chat Mesajları

**Mesaj Listesi:**

```
GET /api/video-streams/{streamId}/messages?since={ISO8601}&limit=50
Headers: Authorization: Bearer {JWT} // opsiyonel
Response: [
  {
    "id": "...",
    "streamId": "...",
    "content": "Merhaba!",
    "createdAt": "2025-01-01T00:00:00.000Z",
    "user": {
      "id": "...",
      "name": "Ali",
      "nickname": null,
      "image": "https://d2gjqh9j26unp0.cloudfront.net/profilepic/79e5807d-f405c987b9f5d860d225970f",
      "role": "free",
      "membership": null
    }
  }
]
```

`since` parametresi ISO8601 formatında. Belirtilmezse son 50 mesaj gelir.

### Mesaj Gönder:

```
POST /api/video-streams/{streamId}/messages
Headers: Authorization: Bearer {JWT}
Body: {
  "content": "Merhaba!",      // zorunlu ("message", "body", "text" alternatifleri de
  "nickname": null,           // opsiyonel, özel takma ad
  "isHidden": false           // opsiyonel, true = anonim mesaj (isim ve resim
                                gizlenir)
}
Response: {
  "id": "...",
  "streamId": "...",
  "content": "Merhaba!",
  "createdAt": "...",
  "user": { "id": "...", "name": "Ali", "image": "...", "role": "free" }
}
```

Mesaj SSE üzerinden de tüm dinleyicilere `streamMessage` olarak iletilir.

## 2.8 Yorum (Eski endpoint — messages ile aynı veri)

```
GET /api/video-streams/{streamId}/comments
POST /api/video-streams/{streamId}/comments
Body: { "content": "...", "nickname": null, "isHidden": false }
```

messages endpoint ile aynı model (VideoStreamComment) kullanılır.

## 2.9 Beğeni

```
POST /api/video-streams/{streamId}/like
Headers: Authorization: Bearer {JWT} // opsiyonel
Body: { "count": 5 } // 1-100 arası, opsiyonel (default: 1)
Response: { "likeCount": 42 }

GET /api/video-streams/{streamId}/like
Response: { "likeCount": 42 }
```

## 3. HEDİYE SİSTEMİ

### 3.1 Hediye Tipleri Listesi

```
GET /api/video-streams/gifts
Response: [
  {
    "id": "...",
    "name": "Gül",
    "nameEn": "Rose",
    "icon": "🌹",
    "animation": null,
    "price": 100,
    "sortOrder": 1,
    "isActive": true
  }
]
```

### 3.2 Hediye Gönder (Video Stream)

```
POST /api/video-streams/{streamId}/gifts
Headers: Authorization: Bearer {JWT}
Body: {
  "giftTypeId": "gift-type-id",
  "quantity": 1 // 1-100 arası, default 1
}
Response: {
  "success": true,
  "gift": {
    "id": "...",
    "streamId": "...",
    "senderId": "...",
    "giftTypeId": "...",
    "quantity": 1,
    "totalPrice": 100,
    "sender": { "name": "Ali", "image": "..." },
    "giftType": { "name": "Gül", "icon": "🌹", "price": 100 }
  },
  "newBalance": 4900,
  "pkUpdate": { "battleId": "...", "score1": 500, "score2": 300 } // sadece PK aktif-
se
}
```

Admin/yönetici rolü olan kullanıcılardan jeton düşülmez (sınırsız bakiye).

PK aktifse hediye otomatik olarak PK skoruna eklenir.

Hediye SSE üzerinden tüm izleyicilere `gift` event'i olarak iletilir.

### 3.3 Son Hediye

```
GET /api/video-streams/{streamId}/gifts
Response: [
  {
    "id": "...",
    "streamId": "...",
    "senderId": "...",
    "giftTypeId": "...",
    "quantity": 1,
    "totalPrice": 100,
    "createdAt": "...",
    "sender": { "name": "Ali", "image": "..." },
    "giftType": { "name": "Gül", "icon": "🌹", "price": 100 }
  }
]
```

## 4. PK BATTLE (Kapışma) — VIDEO STREAM

### 4.1 Aktif PK Getir (Stream bazlı)

```
GET /api/video-streams/pk?streamId={streamId}
Headers: Authorization: Bearer {JWT}
Response: {
  "id": "pk-battle-id",
  "stream1Id": "challenger-stream-id",
  "stream2Id": "opponent-stream-id",
  "user1Id": "challenger-user-id",
  "user2Id": "opponent-user-id",
  "score1": 500,
  "score2": 300,
  "status": "active",      // pending | active | completed | cancelled | rejected
  "duration": 180,        // saniye
  "startedAt": "2025-01-01T00:00:00.000Z",
  "endedAt": null,
  "winnerId": null,
  "createdAt": "...",
  "user1": { "id": "...", "name": "Ali", "image": "..." },
  "user2": { "id": "...", "name": "Veli", "image": "..." },
  "stream1": { "id": "...", "title": "..." },
  "stream2": { "id": "...", "title": "..." }
}
```

PK bulunamazsa `null` döner.

Son 5 dakikada tamamlanmış PK'lar da döner (sonuç ekranı için).

## 4.2 PK Başlat

```
POST /api/video-streams/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "create",
  "streamId": "{myStreamId}",
  "targetStreamId": "{opponentStreamId}",
  "duration": 180 // saniye (default: 180, yani 3 dakika)
}
Response: {
  "id": "pk-battle-id",
  "stream1Id": "...",
  "stream2Id": "...",
  "user1Id": "...",
  "user2Id": "...",
  "status": "pending",
  "duration": 180
}
```

Rakip yayıncıya push notification gönderilir.  
Her iki stream'in de `status: live` olması gerekir.  
Taraflardan biri zaten bir PK'daysa hata döner.

## 4.3 PK Kabul

```
POST /api/video-streams/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "accept",
  "battleId": "pk-battle-id"
}
Response: {
  ...updatedBattle,
  "status": "active",
  "startedAt": "...",
  "endTime": "2025-01-01T00:03:00.000Z" // bitiş zamanı (startedAt + duration)
}
```

Sadece user2 (rakip yayıncı) veya rakip odanın sahibi/moderatörü kabul edebilir.

## 4.4 PK Reddet

```
POST /api/video-streams/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "reject",
  "battleId": "pk-battle-id"
}
Response: { ...battle, "status": "rejected", "endedAt": "..." }
```

## 4.5 PK İptal

```
POST /api/video-streams/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "cancel",
  "battleId": "pk-battle-id"
}
Response: { ...battle, "status": "cancelled", "endedAt": "..." }
```

## 4.6 PK Bitir (Kazanan Otomatik Hesaplanır)

```
POST /api/video-streams/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "end",
  "battleId": "pk-battle-id"
}
Response: {
  ...battle,
  "status": "completed",
  "endedAt": "...",
  "winnerId": "winner-user-id" // score1 > score2 → user1, score2 > score1 → user2,
  eşitlik → null
}
```

## 4.7 Stream-Specific PK Endpoint (Alternatif)

```
GET /api/video-streams/{streamId}/pk-battle
Response: { ...battle with user1, user2, stream1, stream2 }

POST /api/video-streams/{streamId}/pk-battle
Body: {
  "action": "create",
  "targetStreamId": "...",
  "duration": 180
}
// veya
Body: { "action": "accept|reject|cancel|end", "battleId": "..." }
```

## 4.8 PK Skor Güncelleme (Manuel)

```
POST /api/video-streams/pk/score
Headers: Authorization: Bearer {JWT}
Body: {
  "streamId": "{streamId}",
  "amount": 100,
  "side": "challenger" // "challenger" veya "opponent"
}
Response: { "battleId": "...", "score1": 600, "score2": 300 }
```

NOT: Hediye gönderildiğinde skor otomatik güncellenir. Bu endpoint sadece manuel güncelleme için.

## 4.9 Aktif PK Listesi

```
GET /api/video-streams/pk/list
Response: [{ ...battle with user info }]
```

## 5. PK BATTLE — CHAT ROOM (Sesli Sohbet Odaları)

Chat room PK'ları aynı `PKBattle` modelini kullanır. `stream1Id / stream2Id` alanlarına `roomId` yazılır.

### 5.1 Aktif PK Getir (Room bazlı)

```
GET /api/chat/rooms/{roomId}/pk
Headers: Authorization: Bearer {JWT}
Response: {
  "id": "pk-battle-id",
  "stream1Id": "{roomId}",
  "stream2Id": "{roomId}",
  "user1Id": "...",
  "user2Id": "...",
  "score1": 200,
  "score2": 150,
  "status": "active",
  "duration": 180,
  "startedAt": "...",
  "endedAt": null,
  "winnerId": null,
  "user1": { "id": "...", "name": "...", "image": "...", "username": "..." },
  "user2": { "id": "...", "name": "...", "image": "...", "username": "..." },
  "room1": { "id": "...", "name": "Oda Adı", "icon": "🎵" },
  "room2": { "id": "...", "name": "Oda Adı 2", "icon": "🌟" }
}
```

### 5.2 PK Başlat (Sadece Oda Sahibi Yapabilir)

```
POST /api/chat/rooms/{myRoomId}/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "create",
  "targetRoomId": "{opponentRoomId}",
  "duration": 180 // saniye, default 180
}
Response: {
  "id": "pk-battle-id",
  "stream1Id": "{myRoomId}",
  "stream2Id": "{opponentRoomId}",
  "user1Id": "my-user-id",
  "user2Id": "opponent-owner-id",
  "status": "pending",
  "duration": 180
}
```

Rakip oda sahibine push notification gönderilir.

Her iki odanın da `isActive: true` olması gerekir.

SSE üzerinden her iki odaya `pk` event'i gönderilir.

### 5.3 PK Kabul

```
POST /api/chat/rooms/{roomId}/pk
Headers: Authorization: Bearer {JWT}
Body: {
  "action": "accept",
  "battleId": "pk-battle-id"
}
Response: {
  ...updatedBattle,
  "status": "active",
  "startedAt": "...",
  "endTime": "..." // bitiş zamanı
}
```

user2 (rakip oda sahibi), veya rakip odanın moderatörü/admin'i kabul edebilir.  
Challenger'a "PK Kabul Edildi" push notification gönderilir.  
SSE: her iki odaya `pk:started` event.

### 5.4 PK Reddet

```
POST /api/chat/rooms/{roomId}/pk
Headers: Authorization: Bearer {JWT}
Body: { "action": "reject", "battleId": "pk-battle-id" }
Response: { ...battle, "status": "rejected" }
```

### 5.5 PK İptal

```
POST /api/chat/rooms/{roomId}/pk
Headers: Authorization: Bearer {JWT}
Body: { "action": "cancel", "battleId": "pk-battle-id" }
Response: { ...battle, "status": "cancelled" }
```

### 5.6 PK Bitir

```
POST /api/chat/rooms/{roomId}/pk
Headers: Authorization: Bearer {JWT}
Body: { "action": "end", "battleId": "pk-battle-id" }
Response: { ...battle, "status": "completed", "winnerId": "..." }
```

### 5.7 PK Skor Güncelleme (Manuel)

```
POST /api/chat/rooms/{roomId}/pk/score
Headers: Authorization: Bearer {JWT}
Body: {
  "battleId": "pk-battle-id", // opsiyonel, belirtilmezse roomId'ye göre aktif PK bu-
  lunur
  "amount": 100,
  "side": "room1" // "room1" | "room2" | "challenger" | "opponent" | belirtilmezse
  roomId'ye göre otomatik
}
Response: { "battleId": "...", "score1": 300, "score2": 150 }
```

## 5.8 Hediye ile Otomatik PK Skor Güncellemesi

Chat room hediye gönderiminde `streamId` parametresini `roomId` olarak gönderin:

```
POST /api/chat/rooms/{roomId}/gifts
Headers: Authorization: Bearer {JWT}
Body: {
  "giftTypeId": "...",
  "quantity": 1,
  "recipientId": "...",
  "streamId": "{roomId}" // <-- Bu parametre PK skor güncellemesini tetikler
}
Response: {
  "success": true,
  "gift": { ... },
  "pkUpdate": { "battleId": "...", "score1": 400, "score2": 150 } // PK aktifse
}
```

Alternatif: `battleId` ve `side` parametreleri de gönderilebilir.

## 5.9 Tüm Aktif Chat Room PK'ları

```
GET /api/chat/rooms/pk-list?status=active
Headers: Authorization: Bearer {JWT} // opsiyonel
Response: [
  {
    "id": "...",
    "stream1Id": "{roomId}",
    "stream2Id": "{roomId}",
    "user1Id": "...",
    "user2Id": "...",
    "score1": 200,
    "score2": 150,
    "status": "active",
    "user1": { "id": "...", "name": "...", "image": "...", "username": "..." },
    "user2": { "id": "...", "name": "...", "image": "...", "username": "..." },
    "room1": { "id": "...", "name": "...", "icon": "..." },
    "room2": { "id": "...", "name": "...", "icon": "..." }
  }
]
```

`status` parametresi: `active`, `pending`, `completed`, `all`, veya virgülle ayrılmış `active,pending`

## 5.10 PK SSE Events (Chat Room)

SSE endpoint: `GET /api/chat/rooms/{roomId}/sse`

PK event'leri `type: 'pk'` olarak gelir:

```

// PK Oluşturuldu
{"type": "pk", "data": {
  "battleId": "...", "action": "created",
  "room1Id": "...", "room2Id": "...",
  "user1Id": "...", "user2Id": "...",
  "challengerName": "Ali",
  "duration": 180, "status": "pending"
}}

// PK Başladı
{"type": "pk", "data": {
  "battleId": "...", "action": "started",
  "room1Id": "...", "room2Id": "...",
  "score1": 0, "score2": 0,
  "duration": 180, "status": "active",
  "startedAt": "...", "endTime": "...
}}

// Skor Güncellendi
{"type": "pk", "data": {
  "battleId": "...", "action": "score_update",
  "score1": 300, "score2": 150,
  "room1Id": "...", "room2Id": "...",
  "addedAmount": 100, "addedSide": "room1"
}}

// PK Tamamlandı
{"type": "pk", "data": {
  "battleId": "...", "action": "completed",
  "score1": 500, "score2": 300,
  "winnerId": "...", "status": "completed"
}}

// PK Reddedildi/İptal Edildi
{"type": "pk", "data": {
  "battleId": "...", "action": "rejected", // veya "cancelled"
  "status": "rejected"
}}

```

## 6. MİSAFİR DAVET / CO-BROADCAST (Çoklu Yayın)

**Max 8 eşzamanlı co-broadcaster** (invite sub-route'ta limit 4, ana route'ta limit 8).

## 6.1 Co-Broadcaster Listesi

```
GET /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT} // opsiyonel
Response: [
  {
    "id": "co-broadcaster-id",
    "streamId": "...",
    "userId": "...",
    "status": "active",      // invited | active | requested | ended
    "isMuted": false,
    "isVideoOff": false,
    "invitedAt": "...",
    "joinedAt": "...",
    "leftAt": null,
    "user": { "id": "...", "name": "Ali", "image": "..." }
  }
]
```

`status` filtreleri: invited, active, requested (ended olanlar hariç tutulur).

## 6.2 Misafir Davet Et (Sadece Yayıncı)

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "invite", "userId": "{inviteeId}" }
Response: { "id": "...", "status": "invited", ... }
```

### Alternatif Flutter-friendly endpoint:

```
POST /api/video-streams/{streamId}/co-broadcast/invite
Headers: Authorization: Bearer {JWT}
Body: { "inviteeId": "{userId}" }
Response: { "id": "...", "status": "invited", ... }
```

## 6.3 İzleyici Katılma Talebi (Viewer → Yayıncıya İstek)

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "request" }
Response: { "id": "...", "status": "requested", ... }
```

Yayıncıya “Ortak Yayın Talebi” push notification gönderilir.

## 6.4 Yayıncı Talebi Onaylar

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "approve", "userId": "{requesterId}" }
Response: { "id": "...", "status": "active", "joinedAt": "...", ... }
```

Talep eden kişiye “Onaylandı” push notification gönderilir.

## 6.5 Yayınçı Talebi Reddeder

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "reject_request", "userId": "{requesterId}" }
Response: { "success": true }
```

## 6.6 Davet Kabul Et (Davet Edilen Kişi)

```
PATCH /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "accept" }
Response: { "id": "...", "status": "active", "joinedAt": "...", ... }
```

Yayıncıya “Kabul Edildi” push notification gönderilir.

Bu noktada Agora token’ı `role: "host"` olarak yeniden alınmalıdır.

## 6.7 Davet Reddet / Ayrıl (Davet Edilen Kişi)

```
PATCH /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "reject" } // davet reddi
Body: { "action": "leave" } // aktif yayından ayrılma
Response: { "success": true }
```

## 6.8 Misafir Yönetimi (Yayıncı Tarafından)

### Sessize Al:

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "mute", "userId": "{guestUserId}" }
Response: { ...coBroadcaster, "isMuted": true }
```

### Sesi Aç:

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "unmute", "userId": "{guestUserId}" }
Response: { ...coBroadcaster, "isMuted": false }
```

### Kamerayı Kapat:

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "video_off", "userId": "{guestUserId}" }
Response: { ...coBroadcaster, "isVideoOff": true }
```

### Kamerayı Aç:

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "video_on", "userId": "{guestUserId}" }
Response: { ...coBroadcaster, "isVideoOff": false }
```

#### Misafiri Kov:

```
POST /api/video-streams/{streamId}/co-broadcast
Headers: Authorization: Bearer {JWT}
Body: { "action": "remove", "userId": "{guestUserId}" }
Response: { "success": true }
```

## 6.9 Co-Broadcast Akış Şeması

### [DAVET AKIŞI]

```
Yayıncı → invite(userId) → status: invited
  → Davet edilen kişi PATCH accept → status: active → Agora token (host) al → yayına katıl
  → Davet edilen kişi PATCH reject → status: ended
```

### [TALEP AKIŞI]

```
İzleyici → request() → status: requested → Yayıncıya push notification
  → Yayıncı approve(userId) → status:
active → İzleyiciye push → Agora token (host) al → yayına katıl
  → Yayıncı reject_request(userId) → status: ended
```

## 7. MODERATÖR & BAN & MUTE SİSTEMİ

### 7.1 Moderatör Yönetimi (Max 10 moderatör/yayın)

#### Moderatör Listesi:

```
GET /api/video-streams/{streamId}/moderators
Response: [
  {
    "id": "mod-id",
    "userId": "user-id",
    "user": { "name": "Ali", "image": "..." }
  }
]
```

#### Moderatör Ekle (Sadece Yayıncı):

```
POST /api/video-streams/{streamId}/moderators
Headers: Authorization: Bearer {JWT}
Body: { "userId": "user-id" }
Response: { "id": "...", "streamId": "...", "userId": "...", "addedAt": "..." }
```

#### Moderatör Çıkar (Sadece Yayıncı):

```
DELETE /api/video-streams/{streamId}/moderators
Headers: Authorization: Bearer {JWT}
Body: { "userId": "user-id" }
Response: { "success": true }
```

## 7.2 Ban Yönetimi

### Banlı Kullanıcı Listesi:

```
GET /api/video-streams/{streamId}/ban
Response: [{ "bannedUserId": "...", "reason": "...", "user": { "name": "...", "image": "..."} } ]
```

### Kullanıcı Banla (Sadece Yayıncı):

```
POST /api/video-streams/{streamId}/ban
Headers: Authorization: Bearer {JWT}
Body: { "userId": "user-id", "reason": "Spam yapıyor" }
Response: { "id": "...", "streamId": "...", "bannedUserId": "..." }
```

Banlanan kullanıcı co-broadcaster ise otomatik çıkarılır.

### Ban Kaldır (Sadece Yayıncı):

```
DELETE /api/video-streams/{streamId}/ban?userId={userId}
Headers: Authorization: Bearer {JWT}
Response: { "success": true }
```

## 7.3 Mute Yönetimi (Yayıncı veya Moderatör)

### Sessize Alınmış İzleyici Listesi:

```
GET /api/video-streams/{streamId}/mute
Response: ["user-id-1", "user-id-2"] // sadece userId listesi
```

### İzleyiciyi Sessize Al:

```
POST /api/video-streams/{streamId}/mute
Headers: Authorization: Bearer {JWT}
Body: {
  "viewerId": "user-id",
  "reason": "Uygunsuz mesaj", // opsiyonel
  "expiresAt": "2025-01-01T01:00:00.000Z" // opsiyonel, null = süresiz
}
```

### Sessizliği Kaldır:

```
DELETE /api/video-streams/{streamId}/mute
Headers: Authorization: Bearer {JWT}
Body: { "viewerId": "user-id" }
Response: { "success": true }
```

## 8. WebRTC SIGNALING

### 8.1 Stream-Specific Signal

```
GET /api/video-streams/{streamId}/signal?recipientId={userId}
Response: [
  {
    "id": "signal-id",
    "type": "offer|answer|ice-candidate|join",
    "senderId": "...",
    "data": { ... },
    "createdAt": "..."
  }
]
```

Alınan sinyaller otomatik `processed: true` işaretlenir.

```
POST /api/video-streams/{streamId}/signal
Body: {
  "type": "offer|answer|ice-candidate",
  "receiverId": "target-user-id",
  "data": { "sdp": "..." }
}
Response: { "success": true }
```

```
DELETE /api/video-streams/{streamId}/signal
// Eski sinyalleri temizle (60 saniyeden eski)
Response: { "success": true }
```

### 8.2 Genel Signal Endpoint

```
GET /api/video-streams/signal?streamId={id}&recipientId={userId}
POST /api/video-streams/signal
Body: { "streamId": "...", "type": "...", "receiverId": "...", "data": {...} }
```

## 9. AGORA ENTEGRASYON NOTLARI

- **App ID:** `f1cf983a38114b04a4e9102c303ba63e`
- **Token endpoint:** `POST /api/agora/token`
- **Channel Name:** `streamId` kullanılıyor (yayın oluştururken dönen ID)
- **Token süresi:** 24 saat
- **Roller:**
  - Yayıncı (host) = `RtcRole.PUBLISHER` → video/audio yayınlayabilir
  - İzleyici (audience) = `RtcRole.SUBSCRIBER` → sadece izleyebilir
- **Co-Broadcast:** Misafir kabul edilince → audience role'den host role'e geçiş yapılmalı, yeni token alınmalı

- **PK:** İki ayrı Agora channel birleştirilmez. Her yayıncı kendi channel'ında kalır. UI tarafında split-screen gösterilir. Skorlar API üzerinden senkronize edilir.
  - **UID:** 0 gönderilirse Agora auto-assign yapar. Sabit UID kullanmak isterseniz numeric ID gönderin.
-

## 10. DATABASE MODELLERİ (Referans)

---

```

VideoStream {
    id                String      @id @default(cuid())
    userId            String      // yayıncının user ID'si
    title             String?
    description        String?
    status            String      @default("live")    // live | ended
    viewerCount       Int         @default(0)
    likeCount         Int         @default(0)
    roomId            String      @unique @default(cuid())
    category           String?    // fortune | chat | general
    thumbnailUrl       String?
    broadcastImage     String?    // resimli yayın modu
    isImageMode        Boolean    @default(false)
    backgroundUrl      String?
    lastGiftAt        DateTime?  // auto-close için
    autoClosedAt       DateTime?
    startedAt          DateTime  @default(now())
    endedAt            DateTime?
}

StreamCoBroadcaster {
    id                String      @id @default(cuid())
    streamId           String
    userId            String
    status            String      @default("invited") // invited | active | ended | requested
    isMuted            Boolean    @default(false)
    isVideoOff         Boolean    @default(false)
    invitedAt          DateTime  @default(now())
    joinedAt           DateTime?
    leftAt             DateTime?
    @unique([streamId, userId])
}

PKBattle {
    id                String      @id @default(cuid())
    stream1Id          String      // Video stream: streamId, Chat room: roomId
    stream2Id          String      // Video stream: streamId, Chat room: roomId
    user1Id            String      // Challenger user
    user2Id            String      // Opponent user
    score1             Int         @default(0)    // Hediye puanları (jeton cinsinden)
    score2             Int         @default(0)
    status            String      @default("pending") // pending | active | completed | cancelled | rejected
    duration           Int         @default(180)    // saniye
    startedAt          DateTime?
    endedAt            DateTime?
    winnerId           String?    // Kazanan user ID (beraberlikte null)
}

VideoStreamViewer {
    id                String      @id @default(cuid())
    streamId           String
    viewerId           String      // user ID veya "guest_xxx"
    viewerName         String?
    joinedAt           DateTime  @default(now())
    leftAt             DateTime?
    @unique([streamId, viewerId])
}

VideoStreamSignal {
    id                String      @id @default(cuid())
    streamId           String

```

```

senderId    String
receiverId  String?
signalType  String // offer | answer | ice-candidate | join
signalData  String @db.Text
processed   Boolean @default(false)
}

StreamGift {
  id          String @id @default(cuid())
  streamId    String
  senderId    String
  giftTypeId  String
  quantity    Int @default(1)
  totalPrice  Int
}

StreamModerator {
  id          String @id @default(cuid())
  streamId    String
  userId      String
  @@unique([streamId, userId])
}

StreamBan {
  id          String @id @default(cuid())
  streamId    String
  bannedUserId String
  reason      String?
  @@unique([streamId, bannedUserId])
}

VoiceSession {
  id          String @id @default(cuid())
  roomId      String
  userId      String
  userName    String
  agoraUid    Int @default(0)
  isActive    Boolean @default(true)
  lastPing    DateTime @default(now())
  @@unique([roomId, userId])
}

```

## 11. ÖNEMLİ NOTLAR VE İPUÇLARI

1. **Authentication:** Tüm istekler `Authorization: Bearer {JWT}` header'ı ile yapılmalı. Backend dual auth destekler: önce JWT kontrol edilir, yoksa NextAuth session'a bakılır.
2. **SSE Bağlantısı:** Video stream izlerken `GET /api/video-streams/{streamId}/stream` SSE endpoint'ine bağlanın. Bağlantı koptuğunda otomatik yeniden bağlanma mekanizması kurun.
3. **PK Timer:** PK başladığında `startedAt` + `duration` ile bitiş zamanını hesaplayın. Timer client-side çalışır. Süre bittiğinde `action: "end"` çağırın.
4. **PK Skor:** Hediye gönderildiğinde skor otomatik güncellenir. SSE üzerinden `score_update` event'i gelir. Manuel skor güncelleme sadece özel durumlar için.

5. **Co-Broadcast Agora Geçişi:** Bir izleyici co-broadcaster olduğunda:
  - Önce mevcut Agora bağlantısını kapatın (audience)
  - Yeni token alın ( role: "host" )
  - Host olarak yeniden bağlanın
  - Video/audio publish etmeye başlayın
6. **Image Mode:** `isImageMode: true` ve `broadcastImage` URL'si ile kamera yerine resim yayını yapılabilir. Agora'da video track yerine resmi gösterin.
7. **Auto-Close:** 30 dakika hediye gelmezse yayın backend tarafından otomatik kapatılır.
8. **Push Notifications:** PK daveti, co-broadcast talebi/kabulü, yayın başlatma bildirimlerinde OneSignal push gönderilir. Flutter'da notification handler'da bu event'leri yakalayıp ilgili ekrana yönlendirin.
9. **Viewer Count:** Gerçek izleyici sayısı `VideoStreamViewer` tablosundan hesaplanır (leftAt null olanlar). `viewerCount` alanı cache/approximate olabilir.
10. **Error Responses:** Tüm hatalar Türkçe mesajlarla döner:
  - 401: "Oturum açmanız gerekiyor" / "Giriş yapmalısınız"
  - 403: "Yetkiniz yok" / "Sadece yayıncı davet gönderebilir"
  - 400: "Zaten aktif bir PK mevcut" / "Yetersiz jeton"
  - 404: "Yayın bulunamadı" / "PK bulunamadı"
11. **Komisyon Sistemi:** Hediye gönderiminde:
  - Video stream: Platform komisyonu (default %30), geri kalanı yayıncıya
  - Chat room: %50 platform, %50 alıcıya + koltukta oturuyorsa %10 oda sahibine
  - Admin/yönetici gönderimlerinde jeton düşülmez ve alıcıya bakiye yansımaz
12. **Rate Limiting:** Stream listesi endpoint'i 10 saniyelik cache kullanır. Çok sık poll yapmayın.